EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture 05 — October 14, 2025

Author: Jonathan Dutoit

Professor: Mathias Payer

Programming Languages

Learning goals for today

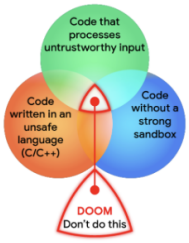
- Understand how modern programming languages achieve key safety properties and their security implications
- Differentiate Type Safety, Memory Safety, and Thread Safety as policies
- How to use Sandboxing and Compartmentalization to handle complex code

Motivation

Google "Two out of Three" Security Rule

Developer may only select 2 of 3 options

Using a "safe"programming languages makes 2-out-of-3 rule unnecessary



Consequences of Memory Safety Vulnerabilities

Memory safety means that an attacker can modify your memory. If you think about the process that is executing code (load, compute, store machine), thus the attacker has the power to modify the memory and make your program do arbitrary things. Memory safety issues are **really severe**.

In 2022, memory-safety issues composed 86% of critical severity security vulnerabilities, highest ranking

Memory Safety Violations are Most Common

A major portion (224/1228 $\sim$ 18%) of bugs found by SyzBot is still memory corruption errors in Linux Kernel.

Memory Safety



76% of Android vulnerabilities in 2019

24% of Android vulnerabilities in 2024

- 70% of Android's high severity security vulnerabilities (2021)
- Android 13 is first Android release where majority of new code is in a memory safe language

Most bugs are in new code: that's why replacing legacy code with their new Rust implementation then just implies we're just going to have much more bugs in Rust. Replacing legacy issues is not the solution. Rust may be a safe language and thus implying no memory vulnerabilities, but rewriting code is complex.

Is a Safe Language Better for Security?

"[The] NSA recommends using a memory safe language when possible. While the use of added protections to non memory safe languages and the use of memory safe languages do not provide absolute protection against exploitable memory issues, they do provide considerable protection."

A safe programming language leaves you more time to handle programming logic than memory or handling other operations aspects.

Three Types of Safety:

- Type Safety
- Memory Safety
- Thread Safety

Type Safety

Is the object an elephant or a crocodile?

Type System

*Logical system that assigns a **type** to every **term** (variable, expression, function)*

Type of a **value** specifies which operations can be applied to it

Type of a **variable** limits values that it can be assigned

Type of an **expression/function** delimits values it accepts and can produce

Many different type systems

- Static/dynamic
- Declared/inferred
- Strict/relaxed

Well-typed programs cannot "go wrong". (Robin Milner)

If a type system is sound, then expressions accepted by that type system must evaluate to a value of the appropriate type (rather than produce a value of some other, unrelated type or crash with a type error). A language is type-safe if the only operations that can be performed on data in the language are those sanctioned by the type of the data.

- Type safety depends on the language
- Influenced by static/dynamic semantics

Static/Dynamic Typing

Static type system is checked by the compiler (or preprocessors) before program executes

- Type checker uses typing rules to determine type of all terms
- Compare inferred types against declared types and operations performed on values
- Mismatch is typing error

Dynamic type system tracks types of objects at runtime and prevents improper operations from being applied to them.

Typing Pros/Cons

Static – checked at compile time

- No runtime overhead (memory or time)
- Find errors before program runs
- Find all errors
- Better for building reliable systems and groups of developers

Dynamic – checked at runtime

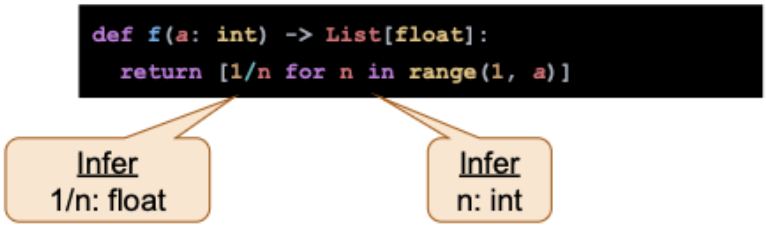
- More flexible (at least without complex type system)
- Fewer type declarations
- Better for quick dirty programming ("scripting")

Declared/Inferred Types

Languages previously required full type declarations for all functions and variables. Compilers only infer expression types.

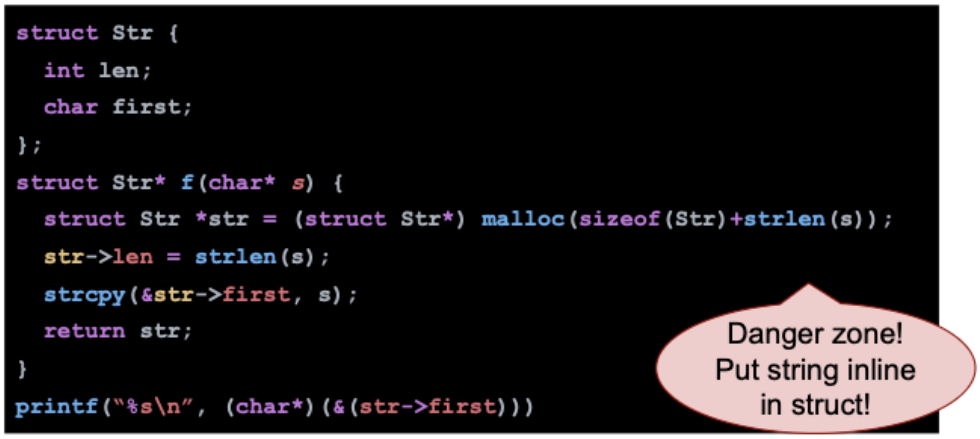
Hindley-Milner type inference (Algorithm W) used in Haskell. Later spread to other languages.

- Declare function argument and result type
- Compiler infers types in function body
- Scala, C, Java (v8), Typescript, ...



Strict/Relaxed (or Weak)

C is not strict! It "trusts" programmer-supplied type conversions.



What's Up with C's Type System?

C was a thin layer over assembly language, designed by talented programmer for talented programmer

- Machines were slow. Memory was tight

Type system did not constrain programmer or prevent them from using assembly language tricks

Clearly successful, but times change

- Correctness and security are far more important, and C's type system it not helpful

Type Safety and Security

Type checking is the only "automatic" bug detection technique in widespread use

- Type system might prevent you from expressing something exactly in the manner you want
- But it will not stop you from expressing a computation:
  Language with type system is still Turing-complete
- Problems found by sound type system are real errors: No false positives

Bugs do not always cause security problems, but they can

Type system can enforce rules that enhance security

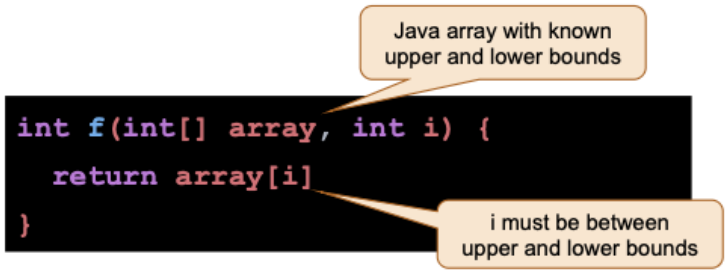
Memory Safety

Is the monkey caged or free?

Memory Safety

Memory safety is an essential programming language property

Code can only access data within live regions of memory whose pointer is properly obtained



Memory safety ensures that only valid objects are accessed in bounds

Spatial memory safety: objects are only accessed in bounds

- After pointer arithmetic, the new pointer still points to the same object

Temporal memory safety: only valid objects are accessed

- The underlying object has not been freed

Memory Corruption

Unintended modification of memory location due to missing / faulty safety check

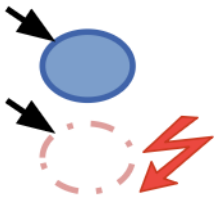
```
void vulnerable(int user1, int *array) {
   // missing bound check for user1
   array[user1] = 42;
}
```

Spatial Memory Safety Error

The pointer is updated to point outside of the valid object

The pointer is used to dereference invalid memory

```
void vulnerable() {
   char buf[12];
   char *ptr = buf[11];
   *ptr++ = 10;
   *ptr = 42; // Out of bound write!
}
```



Temporal Memory Safety Error

The referenced object is freed and no longer ”live”. The pointer is used to dereference invalid memory. ”Use after free”

Memory Safety: Implementation

Bounds checking

- Check that every indexed memory reference is within array bounds
- Compiler optimizations can eliminate some runtime checks

Lifetime tracking

- Use after free is prohibited (e.g., by trapping accesses)
- Double free is prohibited (e.g., by delaying frees)
- Memory leakage is bad (but allowed)

Automated storage reclamation

- Garbage collection
- Reference counting
- Smart pointers/borrow checking

Spatial Memory Safety: Array Access

Every region of memory has a length associated with it

- C's interior pointers make tracking more complex
- Programmer needs to verify that every memory access is in-bounds manually
- Almost every other modern PL automatically inserts bound check for array access

```
int array_access(int array[], int size, int index) {
    if (index > size)
        return 0;
    else
        // *(array + i * sizeof(int))
        return array[index];
}
```

Rust Bounds Checking

```
pub fn access_slice(
    bytes: &[u8],
    index: usize
) -> u8 {
    return bytes[index];
}
```

Rust automatically insert bounds check for slices. Aborts when index is out of bound.

```
access_slice:
        cmp     rdx, rsi
        jae     .out_of_bound
        movzx   eax, byte ptr [rdi + rdx]
        ret
.out_of_bound:
        push    rax
        lea     rax, [rip + .L__unnamed_1]
        mov     rdi, rdx
        mov     rdx, rax
        call    qword ptr [rip +
core::panicking::panic_bounds_check@GOTPCREL]
```

Overhead of Bounds Checking

Overhead for enforcing spatial memory safety for C code is significant

- Softbound reporter 67% overhead

- Unfortunately, C's arbitrary pointer arithmetic, conflation of pointers and arrays, and programmer-visible memory layout make retrofitting C/C++ with spatial safety guarantees extremely challenging."

Overhead for Java is not nearly as large

- 3-10% on small benchamarks
- found roughly the same for Rust

Automated Storage Reclamation

Reclaim and reuse memory that will not be accessed in future execution

- Conservative approximation: reclaim when memory is not reachable

Prevent errors with manual reclamation (malloc/free)

- Double free
- Use after free
- (not freeing)

Many algorithms (!)

- Reference counting
- Garbage collection

Temporal Memory Safety and Security

Cool lab/project: implementing my own memory allocator (managing heap)

Double free is an **integrity** concern

- libc's memory allocator stores **metadata** alongside allocated objects.
- Freeing memory chunks twice will **corrupt** these metadata.
- The corrupted metadata can lead to two different pointers pointing to the same memory location. If an attacker controls one of those pointers, they can change the contents of that memory, which can cause security issues or even allow them to execute code.

Missing free is an **availability** concern

- Basis for denial-of-service attacks (Out-of-Memory)

Reference Counting

Every block of memory maintains a count of numbers of pointers to it

- One when block allocated
- Copying pointer increments count
- Dropping pointer decrements count

Conceptually simple, but fidgety to get right (try it)

Memory freed immediately when last pointer is dropped (count = 0)

- No pauses (unlinke Garbage Collection)
- Incr/Decr overhead can be significant

Count never reaches zero in graphs with cycles.

Language implementations: Python, PHP

Garbage Collection (GC)

When program is running out of free memory, stop and run collector to find unreachable objects

Pauses program execution

Uncollected garbage increases memory requirements

May be less expensive than RC

Collects unreachable cycle structures

Language implementations: JavaScript (v8, SpiderMonkey), Ruby, Lua

Most common implementation of GC: *Mark and Sweep*, made with a `Root Set` and marking objects that have memory references, and sweeping all that don't in the second phase.

GC overhead

Garbage collection overhead is difficult to measure

- What is baseline? No allocation? Malloc/free? Other GC?
- GC affects behaviour of caches and program performance

Has a reputation for being significant

- Depends on how program is written as well

Most troublesome is unpredictable interruptions

- There are pause-free GCs

C++ Smart Pointers

AS a intermediary step, C++ introduced the notion of "Smart Pointers", for automatic memory management. (Note: not mandatory to use them.)

Smart pointers automatically free their referenced content when it goes out of scope (or refcount reaches 0)

```cpp
class SmartPtr {
   int* ptr; // Actual pointer
public:
   explicit SmartPtr(int* p = NULL) {
     ptr = p;
   }
   // Destructor
   ~SmartPtr() { delete (ptr); }
   // Overload dereferencing operator
   int& operator*() { return *ptr; }
};
```

```cpp
int main()
{
   SmartPtr ptr(new int());
   *ptr = 20;
   cout << *ptr;

   // We don't need to call delete ptr
   // when the object ptr goes out of scope,
   // the destructor for it is automatically called
   // and destructor does delete.

   return 0;
}
```

Rust

Safe programming language (type and memory). Meant for low-end and operating systems as an alternative for C or low-level languages.

- Ownership types reduce need for runtime garbage collection
- Allows unsafe code

For low-level programming

- Efficient code, no garbage collection, optional reference counting

Becoming popular for systems programming

- Already in use for Linux kernel modules

Rust Ownership

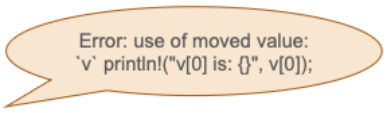
Variables **own** the value they are bound to (contain)

When variable goes out of scope, value is deallocated

Key restriction: exactly **one** variable can be bound to a value

A step forward from C++ smart pointers, as the compiler strictly enforces the constraint. In C++, developers can still choose to use raw pointers.

```rust
let v = vec![1, 2, 3];
let v2 = v;
println!("v[0] is: {}", v[0]);
```



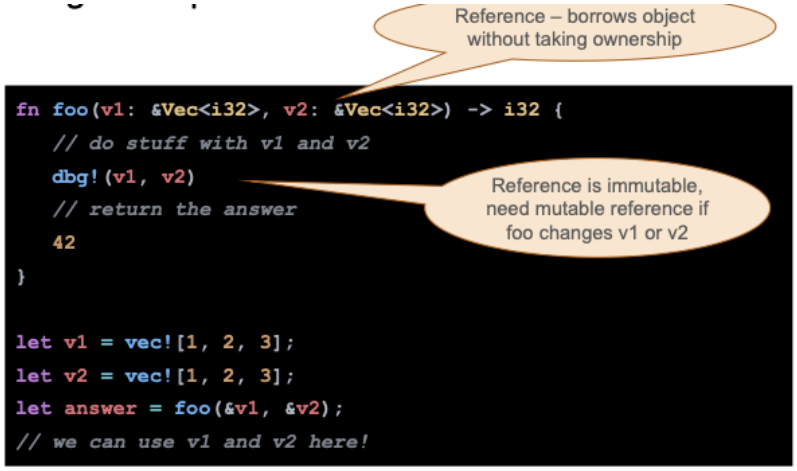
Rust "Borrowing"

Ownership is very restrictive rule that often gets in the way

- How do we pass a value to a function?

"Borrowing" handles common situations by sharing ownership between two variables so long as

- Borrower's lifetime is no longer than owner's lifetime
- Either one writer or multiple readers



Cannot have mutable and immutable borrow to a value at the same time

Rust Ownership Pain

"Implemented a linked list" is a meme in the Rust community.

- People wrote an 8-chapter book on implementing linked lists in Rust.
- A singly linked list is already quite verbose. Need to use `Option` and `Box`
- A doubly linked is challenging due to cyclic reference of the previous node and next node.

Unsafe Code

Rust also allows unsafe code that would not pass borrow checker

Put unsafe code in module and wrap in safe interface

Almost all non-trivial data structures implemented this way

- Linked lists, Trees
- What does that say about type system?

Easier to understand and hand-verify a small amount of code that is explicitly marked unsafe

Rewrite Everything in Rust?

In many cases, simply not possible

- Chromium core is over 33 million lines of code. (Mostly C++)
- The Linux kernel is around 30 million lines of code (Mostly C)
- Simply too much to rewrite

For old code

- Employ fuzzing, mitigation and hardening (Defense in-depth)

For newer code

- Prefer safe language like Rust (Linux kernel modules, Android binder have been rewritten or new code is not Rust)

Clean state not needed, simply writing new code in Rust suffices!

Thread Safety

Code runs in parallel and may access the same data structures faster (or slower)

Thread Safety

Thread-safe code only manipulates shared data structures in a manner that ensures that all threads behave properly and fulfill their design specifications without unintended interaction.

Examples of concurrency bugs

- Race Condition: When two or more threads attempt to access and modify shared data simultaneously without proper synchronization, it can lead to inconsistent data states.
- Deadlock: Deadlocks occur when two or more threads are blocked forever, each waiting for the other to release a resource.

Dirty CoW

A local **privilege escalation** attack that exploits a **race condition** in the Linux Kernel memory-management subsystem.

CoW: Copy on Write, a mechanism for efficiently sharing modifiable data. Used extensively in the `mmap` and `fork` syscall.

With the right timing, a read-only mapping of a file can be turned into a writable mapping.

Thread Safety in Programming Languages

Modern languages include type-safe synchronization primitives in their standard lib:

- Locks
- Semaphores
- Atomic Variables, ...

Other thread-safe language constructs

- Java: `synchronized` keyword
- Go: goroutines and channels
- Rust: owernership

Modern languages makes it easier to write thread-safe program, but ultimately it's the programmer's responsibility to ensure the correctness of concurrent programs.

Thread Safety in Java

In java, the `synchronized` keyword is used in the declaration of a method or code block to acquire the mutex lock for an object while the current thread executes the code.

```
class Counter {
   private int i = 0;
   public synchronized void inc() {
       i++;
   }
}
```

Thread Safety in Go

Don't communicate by sharing memory, share memory by communicating

Goroutines: threads created by the go runtime, cheap and fast.

Channels: means of communication between goroutines. Data is **copied** from one end of the channel to another.

It's possible to send pointers, but that once again may create issues.

Thread Safety in Rust

The same mechanism that guarantees temporal memory safety also ensures thread safety: Ownership

Recall in Rust borrowing:

- Owner's lifetime must outlast borrowers lifetime ⇒ No use-after-free
- Either one write or multiple readers ⇒ no data races

If you violate this, you get a **compilation error!**

Sandboxing Compartmentalization

Principle of Least Privilege

The principle of least privilege ensures that a component has the least privileges needed to function

- No extra functionality is present

- Any additionally removed functionality would impact (some) functionality
- Any added functionality would not increase (any) functionality

Sandboxing

Sandboxing implements *principle of least privilege* by running programs (often untrusted code) in an isolated, restricted environment

Sandboxing is usually a implementation feature of programming languages instead of design feature. Depending on the context/application, a script may or may not be sandboxed.

Implemented by restricting access to certain library/system calls

- `secomp` (SeLinux)
- `cgroup` (Docker)
- `rlbox` (Firefox)

Implementation "only" at the language level is challenging

Sandboxing Example

JavaScript/Web Assembly inside the browser

Without explicit permission from the user, opening a web page or document shouldn't leak secrets in your local hard disk.

The premise of sandboxing providing security relies entirely on the implementation of the sandbox. Bugs may still lead to sandbox breaches.

Sandboxing: Hard to Get Right

Flash and VBA macros used to be security nightmares (now phased out)

JavaScript engine bugs remain a source of critical browser vulnerabilities.

Compartimentalization

Compartmentalization can enforce **principle of least privilege**

Break a comlex system into small components

Limit the access of entities to only what is necessary

Prevent error propagation in the system to diminish impact

Compartmentalization: Chromium

Sandboxed render engines: cannot affect the world, except via the exposed API

- Start process, establish IPC channel
- Drop all access privileges
- Do not require admin rights

If a webpage in a tab crashes, it should not bring other tabs or the entire browser down!

Browser kernel API: decide how render engines influence the outside world

- User interaction: display rendered bitmaps, forward input events
- Storage: manage cookies, passwords, authorize uploads, restrict downloads
- Network: handle HTTP requests and responses, restrict certain schemes (e.g., file://)

Compartmentalization in PL

- What should a basic entity be: functions, libraries? (component)
- How should each component interact with each other? (policy)

Summary

Memory safety issues remain dominant. Modern PLs achieve memory safety through bounds checking, lifetime tracking and automatic storage reclamation.

A programming language's type system can be static/dynamic, inferred/declared, strict/relaxed. A well-designed type system prevents type confusion bugs.

Modern languages provide synchronization primitives and incorporate thread-safe constructs in their design

Sandboxing/compartmentalization can enforce least privileges.